UNIT-4

Classification and Regression

Introduction Classification and Regression

Supervised learning tasks are broadly divided into:

- **Classification**
- **Regression**

Both are used for prediction but differ in the type of output produced.

- **Classification** is a supervised learning technique used to assign data into predefined categories or classes based on input features.
- Output is categorical (discrete).
- Model learns patterns from labeled data.
- Uses decision boundaries to separate classes.

Examples

- Email → Spam or Not Spam
- Disease diagnosis → Positive or Negative
- Image recognition → Cat, Dog, Bird

Types of Classification

1. **Binary classification** – Two classes (Yes/No).
2. **Multiclass classification** – More than two classes.
3. **Multilabel classification** – Multiple labels per sample.

Common Algorithms

- Logistic Regression
- Decision Trees
- Naïve Bayes
- Support Vector Machines
- Neural Networks

Applications

- Medical diagnosis
- Fraud detection
- Speech recognition
- Document classification

- **Regression** is a supervised learning method used to **predict continuous numerical values** by modeling relationships between variables.
- Output is continuous numerical value.
- Predicts quantitative results.
- Measures relationship between dependent and independent variables.

Examples

- House price prediction
- Temperature forecasting
- Stock price prediction
- Sales forecasting

Types of Regression

1. Linear Regression
2. Polynomial Regression
3. Ridge/Lasso Regression
4. Support Vector Regression

Applications

- Finance forecasting
- Weather prediction
- Engineering analysis
- Business analytics

Feature	Classification	Regression
Output	Categorical labels	Continuous numeric values
Goal	Identify class/group	Predict numerical quantity
Example	Spam detection	House price prediction
Algorithms	Logistic regression, SVM	Linear regression, SVR
Evaluation	Accuracy, Precision, Recall	RMSE, MAE, R^2

K-Nearest Neighbor (KNN) Algorithm

- K-Nearest Neighbor (KNN) is a **supervised machine learning algorithm** used for both **classification and regression.**
- It predicts the output of a new data point based on the outputs of its nearest neighbors in the training dataset.
- It is called a **lazy learning algorithm** because it does not build an explicit model during training; instead, it stores all training data and performs computation during prediction.

The main principle of KNN is:

- Find **K closest data points (neighbors)** to a new input.
- Assign the class (classification) or value (regression) based on those neighbors.

Closeness is usually measured using **distance metrics**.

Steps of the KNN Algorithm

1. Choose the number of neighbors **K**.
2. Calculate the distance between the new data point and all training points.
3. Identify the K nearest neighbors.
4. For classification → choose the majority class.
5. For regression → compute the average value.

Distance Measures

Common distance metrics include:

Euclidean Distance

$$d = \sqrt{\left(x_1 - y_1\right)^2 + \left(x_2 - y_2\right)^2}$$

Problem:

Classify whether a fruit is **Apple or Orange** based on:

- Weight
- Color score

If a new fruit has weight **160 g** and color score **0.7**,
KNN finds the nearest fruits in the dataset and assigns the most common class.

Advantages

- Simple and easy to implement.
- No training phase required.
- Works well with small datasets.
- Can handle classification and regression.

Disadvantages

- Slow for large datasets.
- Sensitive to noise and irrelevant features.
- Requires proper choice of K value.
- Needs feature scaling.

Applications

- Pattern recognition
- Recommendation systems
- Medical diagnosis
- Image classification
- Fraud detection

```python
import pandas as pd
import numpy as np

data = pd.read_csv("studentperformance.csv")

# Features and labels
X = data[['math score', 'reading score', 'writing score']].values
y = data['result'].values

# New student data
new_student = np.array([65, 60, 62])

# Number of neighbors
k = 3
# Euclidean distance function
def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))
```

```python
# Calculate distances
distances = []
for i in range(len(X)):
    d = euclidean_distance(new_student, X[i])
    distances.append((d, y[i]))

# Sort by distance
distances.sort(key=lambda x: x[0])

# Get k nearest neighbors
neighbors = distances[:k]

# Majority voting
votes = [neighbor[1] for neighbor in neighbors]
prediction = max(set(votes), key=votes.count)

print("Prediction:", "PASS" if prediction == 1 else
 "FAIL")
```

- Dataset is loaded from CSV.
- Distance between new student and all students is calculated.
- Nearest K = 3 students are selected.
- Majority result determines prediction.